

SWE 550.01 SOFTWARE QUALITY ASSURANCE & RELIABILITY

Performance Testing as Code

A Comprehensive Guide to Grafana k6: Theory, Architecture, and Live
Demonstration.

Yusuf

Savaş

Part 1: The Theory

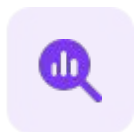
Understanding the "Why" and "How" of modern load testing.

The Problem: Why Load Test?



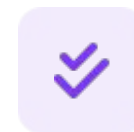
Prevent Outages

Ensure your system remains reliable and available under severe stress and sudden traffic spikes before they reach production.



Find Bottlenecks

Identify slow database queries, memory leaks, or inefficient code paths that only reveal themselves under concurrent load.



Ensure Reliability

Validate that new deployments meet Service Level Objectives (SLOs) and maintain high performance standards continuously.

The **k6** Philosophy

-  **Testing as Code:** Tests are authored in JavaScript/TypeScript, making them native to developers.
-  **Bridging the Gap:** Eliminates silos between Development, QA, and Operations teams.
-  **High-Performance Engine:** Executes JS via a highly optimized Go engine for massive scalability.
-  **CLI-First:** Designed for the terminal, making execution rapid and natural for engineers.



Core Concepts

Virtual Users (VUs)

VUs act as independent threads executing your test script simultaneously. They run in an endless loop, hitting the target APIs sequentially, complete with realistic think times (`sleep()`) to accurately simulate human behavior.

The 4 Golden Signals

k6 automatically measures core metrics out-of-the-box:

Latency: Response time (`http_req_duration`)

Traffic: Throughput (`http_reqs`)

Errors: Failed requests

Saturation: System utilization.

Part 2: Live Demo

Terminal & Editor

Walkthrough

Demo 1: The Baseline

- ▶ **The Smoke Test:** Importing `http` and `sleep` to establish our baseline script.
- 👤 **Single Virtual User:** Running with just 1 VU to verify the endpoint is functional.
- > **Terminal Output:** Understanding metrics like latency (`http_req_duration`) and throughput.

```
import http from 'k6/http';
import { sleep } from 'k6';

export default function () {
  http.get('https://test.k6.io');
  sleep(1);
}
```

Demo 1: Results

```
yusufss@LAPTOP-7M8I4S0R:~/git/k6-test$ k6 run scripts/01-smoke.js
```



```
execution: local
  script: scripts/01-smoke.js
  output: -
```

```
scenarios: (100.00%) 1 scenario, 2 max VUs, 38s max duration (incl. graceful stop):
  * default: 2 looping VUs for 8s (gracefulStop: 30s)
```

TOTAL RESULTS

HTTP

```
http_req_duration.....: avg=74.97ms min=5.67ms med=79.38ms max=147.55ms p(90)=143.03ms p(95)=144.55ms
  { expected_response:true }...: avg=74.97ms min=5.67ms med=79.38ms max=147.55ms p(90)=143.03ms p(95)=144.55ms
http_req_failed.....: 0.00% 0 out of 28
http_reqs.....: 28 3.073174/s
```

EXECUTION

```
iteration_duration.....: avg=1.21s min=1.14s med=1.15s max=1.62s p(90)=1.48s p(95)=1.62s
iterations.....: 14 1.536587/s
vus.....: 2 min=2 max=2
vus_max.....: 2 min=2 max=2
```

```
default ✓ [=====] 2 VUs 8s
```


Demo 2: Ramping Up

```
export const options = {  
  stages: [  
    { duration: '30s', target: 50 },  
    { duration: '1m', target: 50 },  
    { duration: '30s', target: 0 },  
  ],  
};
```

-  **Realistic Traffic:** Using the `options` object to model variable load stages.
-  **Ramp-Up:** Gradually increasing VUs prevents artificial spikes that trigger DDoS protection.
-  **Steady State:** Holding the load to observe system stability and memory leaks over time.

Demo 2: Results

```
yusufss@LAPTOP-7M8I4S0R:~/git/k6-test$ k6 run scripts/02-load-stages.js
```

```
execution: local
  script: scripts/02-load-stages.js
  output: -
```

```
scenarios: (100.00%) 1 scenario, 50 max VUs, 4m0s max duration (incl. graceful stop):
  * default: Up to 50 looping VUs for 3m30s over 3 stages
```

■ TOTAL RESULTS

HTTP

```
http_req_duration.....: avg=78.28ms min=4.47ms med=130.67ms max=715.7ms p(90)=153.93ms p(95)=160.03ms
  { expected_response:true }...: avg=78.28ms min=4.47ms med=130.67ms max=715.7ms p(90)=153.93ms p(95)=160.03ms
http_req_failed.....: 0.00% 0 out of 14340
http_reqs.....: 14340 66.801939/s
```

EXECUTION

```
iteration_duration.....: avg=1.15s min=1.13s med=1.15s max=2.5s p(90)=1.16s p(95)=1.17s
iterations.....: 7170 33.40097/s
vus.....: 1 min=1 max=50
vus_max.....: 50 min=50 max=50
```

NETWORK

```
data_received.....: 27 MB 124 kB/s
data_sent.....: 823 kB 3.8 kB/s
```

```
default ✓ [=====] 00/50 VUs 3m30s
```

Demo 3: Reliability & Checks

- ✔ **Assertions:** Using `check()` to verify HTTP status codes and response bodies.
- ⓘ **Non-Blocking:** Unlike unit tests, a failed check doesn't stop the load test. It tracks error rates.
- ⓘ **Thresholds:** Defining hard pass/fail criteria for your metrics (e.g. $p(95) < 300\text{ms}$).

```
import { check } from 'k6';
import http from 'k6/http';

export const options = {
  thresholds: {
    http_req_duration: ['p(95)<300'],
  },
};

export default function () {
  const res = http.get('https://test.k6.io');
  check(res, {
    'is status 200': (r) => r.status === 200,
  });
}
```

Demo 3: Threshold Passed

```
yusufss@LAPTOP-7M8I4S0R:~/git/k6-test$ k6 run scripts/03-checks-threshold-pass.js
```

```
  execution: local
    script: scripts/03-checks-threshold-pass.js
```

```
  scenarios: (100.00%) 1 scenario, 10 max VUs, 50s max duration (incl. graceful stop):
    * default: 10 looping VUs for 20s (gracefulStop: 30s)
```

THRESHOLDS

```
http_req_duration
✓ 'p(95)<300' p(95)=162.35ms
```

TOTAL RESULTS

```
checks_total.....: 170      8.271667/s
checks_succeeded...: 100.00% 170 out of 170
checks_failed.....: 0.00%   0 out of 170
```

```
✓ status is 200
```

```
HTTP
http_req_duration.....: avg=79.75ms min=4.82ms p(90)=156.97ms p(95)=162.35ms
http_req_failed.....: 0.00% 0 out of 340
http_reqs.....: 340      16.543334/s
```

```
default ✓ [=====] 10 VUs  20s
```

Demo 4: Breaking the Pipeline

```
// Intentionally breaking the threshold  
export const options = {  
  thresholds: {  
    http_req_failed: ['rate<0.01'],  
    http_req_duration: ['p(95)<10'], // Set unre  
  },  
};
```

-  **CI/CD Gates:** Breaking the build pipeline if performance degrades below acceptable limits.
-  **Finding Errors:** If latency spikes beyond 10ms, the test fails instantly.
-  **Shift-Left:** Prevents unoptimized code from reaching the main branch.

Demo 4: Threshold Failed

```
yusufss@LAPTOP-7M8I4S0R:~/git/k6-test$ k6 run scripts/03-checks-threshold-fail.js
```

```
  execution: local
```

```
    script: scripts/03-checks-threshold-fail.js
```

```
  scenarios: (100.00%) 1 scenario, 10 max VUs, 50s max duration (incl. graceful stop):
```

```
    * default: 10 looping VUs for 20s (gracefulStop: 30s)
```

THRESHOLDS

```
  http_req_duration
```

```
  X 'p(95)<10' p(95)=157.26ms
```

TOTAL RESULTS

```
  checks_succeeded ... : 100.00% 174 out of 174
```

```
  ✓ status is 200
```

```
  HTTP
```

```
  http_req_duration.....: avg=79.28ms min=5.53ms p(90)=155ms p(95)=157.26ms
```

```
  http_req_failed.....: 0.00% 0 out of 348
```

```
default ✓ [=====] 10 VUs  20s
```

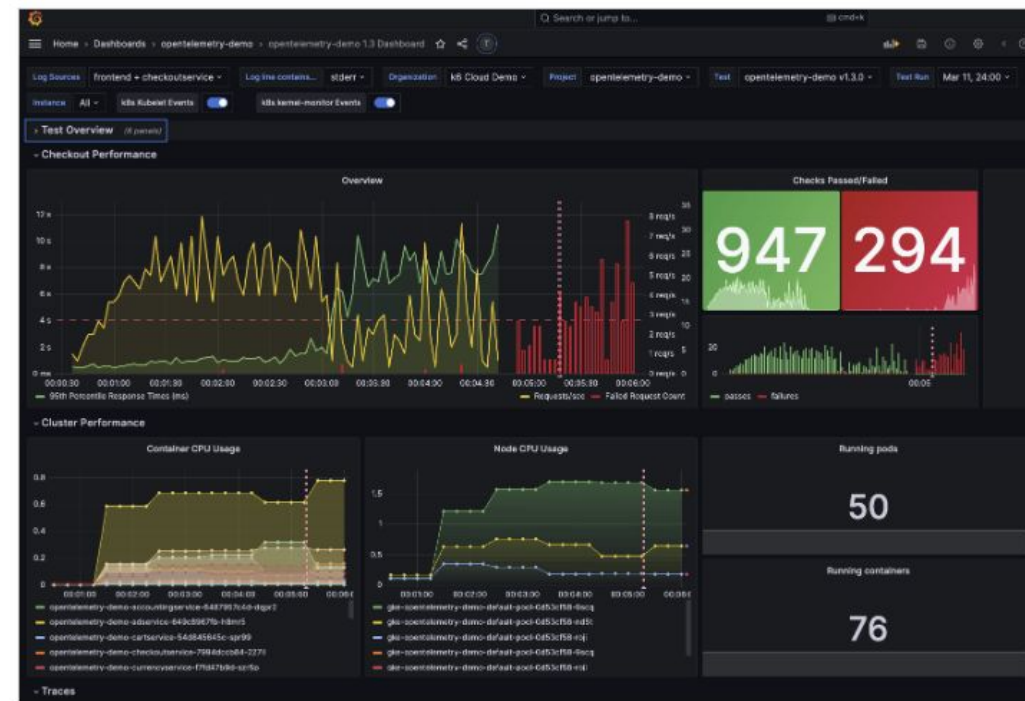
```
ERRO[0021] thresholds on metrics 'http_req_duration' have been crossed
```

Real-time Observability

 **Beyond the CLI:** While terminal output is great for local dev, large scale tests require robust visualization.

 **Native Integrations:** k6 seamlessly streams metrics via `--out` flags to time-series databases.

 **Grafana Dashboards:** Correlate k6 client-side metrics with server-side infrastructure metrics in a single pane of glass.



Wrap-up & Q&A

grafana.com/docs/k6 | github.com/grafana/k6